

Python Bytes Activity Packet

This little booklet will give you some Python to think about and code to write in between club meetings. If you would like more challenges: Talk to the coaches!

Claim A Prize

Python Bytes introduces a new feature that encourages you to code at home: Tier 1 prizes. To claim your Tier 1 prize simply code for at least an hour on your own (not during club meetings!)

Then come to the club meeting on a Wednesday night and tell us how it went. That's it! Now you can claim a Tier 1 prize from among the prize assortment.



But wait! If there is a Tier 1 then there must also be a Tier 2, right? Tier 2 prizes are one tier more luxurious. To claim a Tier 2 there are three steps. First you must agree with one of the coaches on a good project for you. This requires some planning ahead on your part and an all-important coach chat. The second step: Put effort into the project in your spare time to get it working. The third step is: Bring your resulting code to

the club and show the coach how it works. That's it. Offer good while Tier 2 prizes last! And between you and me: The real reward is the project.

Gargutron

Introducing a game we placed on the internet specifically for Python Bytes students to play: **Gargutron!** To play: open a browser and type in the website

<https://pythonbytes.azurewebsites.net/api/Gargutron>

Then follow directions. Hint: The first step is to add some more text to the link above. Gargutron will tell you how. Now on to Python: Playing Gargutron does not count as Tier 1 Prize effort because playing Gargutron is its own reward. However you can definitely write Python code that plays the Gargutron game and in that way earn a Tier 1 prize. Here is how you go about it.

Gargutron code step 1: Check **requests**

Write a one-line program and run it to make sure you have the Python **requests** library available. The one line of code is simply:

```
import requests
```

If this runs without complaining: Continue. If it gives you an error: You must install the `requests` library.

Gargutron code step 2: Set up an `input()` to format a guess

As you know you can input a string from a human being using a line of code like this:

```
guess = input('Input a guess: ')
```

Now you want to format a URL by adding two strings together. The first string is all the stuff leading up to the guess... and the second string is the guess. So this will look like this:

```
url = "https://pythonbytes.azurewebsites.net/api/Gargutron?food=" + guess
```

Now you can pass this url (a string) to the `requests.get()` method:

```
reply = requests.get(url)
```

This variable `reply` is called a *Reponse object*. This Response object contains information about what Gargutron had to say in reply to your message. You have used the `requests` library to `get` a Response from Gargutron. Now we unpack this Response.

Gargutron code step 3: Print Gargutron's reply

Gargutron's reply should be stored in `reply` (from the previous step) as text; so we can look at that by using a print statement like this:

```
print(reply.text)
```

The `.text` is the part of Gargutron's Response for us to read. Now when the program runs: We send some sort of food to Gargutron and find out whether or not he can eat it.

Gargutron code step 4: Put the guesser in a loop

We can put the `input()` - `requests.get()` - `print()` code inside of a loop so that it will keep trying different ideas. Gargutron will start to give hints about what he considers to be good food; so by repeating tries you create a little game.

If you manage to write this code and win the Gargutron game (or if you get stuck): Let us know! (And use that effort to claim a Tier 1 prize if you like.)

Coding Basics

On this page are five programs for you to write. Use these to check your Python skills so far. If you get stuck: No worries! That's what the Python Bytes club and the coaches are for.

Coding Exercise 1: A simple mad-lib. Use the Python built-in function `input()` to ask for five pieces of information: A *name*, a *food item*, a *number*, a *toy*, and an *occupation*. Then print the mad-lib, filling in the blanks from the inputs: `One day _____ was finishing up a _____ when they had a great idea! They entered a nearby skyscraper, ran up the _____ stairs to the top and threw a _____ off the roof. It floated back down on a gust of wind and lightly bumped into a _____. No injuries were reported.`

Coding Exercise 2: How many characters? Define a string variable called `s` to have the value `"On day one I saw a bee."` Use the built-in function for length, called `len()`, to print the number of characters in this string. Your program should print out `"The string s contains _____ characters."`

Coding Exercise 3: Divisible by 17. Write a program that counts from 1 to 300. For each number: Check to see if each number is divisible by 17. If it is divisible by 17: Print the message `"_____ is divisible by 17."` You may find it helpful to use the remainder operator. In python this is the percent sign `%`. You can see how this remainder operator works by running the Python code `print(9 % 3, 9 % 2, 9 % 5)`.

Coding Exercise 4: Fibonacci. Write a program that calculates and prints the first 50 numbers of the Fibonacci sequence. The first seven of them are: 0, 1, 1, 2, 3, 5, 8.

Coding Exercise 5: Random falling. Use the import line `from random import randint` to make the 'random integer' function available to your program. Create a string which is a random number of spaces followed by the letter X. You can do this by simply multiplying the space character `" "` by an integer like `7` and then adding `+ "X"`.

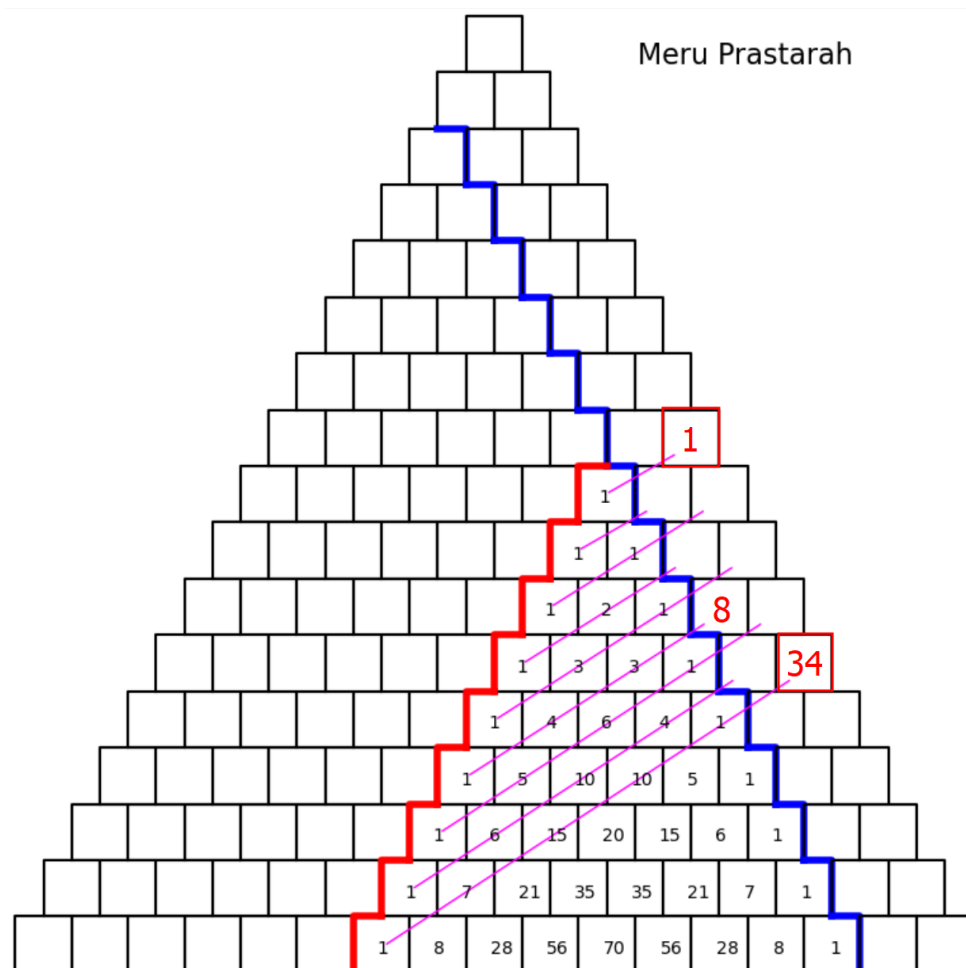
Remember: These exercises are a good check of your Python skills. If you are not sure how to do something: Ask a coach for a little help.

Meru Prastarah

Below is a drawing of Mount Meru, the mystic mathematical mountain at the center of the universe (so I am told). This mountain has other names as well (including “the Pascal triangle”) and fortunately for us the mountain has stairs built in so we can go up and down as we like.

Programming challenge: Create a program that prints the black numbers shown in the diagram below. Do not try to draw the squares; just numbers in triangle. Start with the 1 at the top. Below that the next row will have two ones '1 1' and row three will be '1 2 1'.

Extra challenge: Three numbers are filled in on the right slopes of Mount Meru in red ink. They are 1, 8 and 34. Figure out how these numbers are connected to the numbers in black; and then fill in the missing numbers. Hint: The diagonal lines (magenta) are a *guide*.



Extra Extra Challenge: Can you figure out a way of continuing the red numbers all the way up to the top of the diagram?

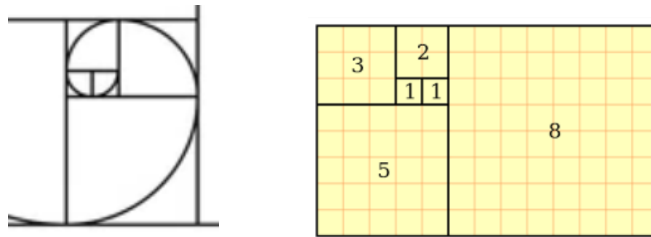
Fibonacci ratios

If we start Fibonacci with 1, 1 then we get 1, 1, 2, 3, 5, 8, 13, 21, 34 and so on. Suppose we use these numbers to make a sequence of fractions: $1/1$, $2/1$, $3/2$, $5/3$, $8/5$, $13/8$, $21/13$, $34/21$ and so on. Let's calculate some Fibonacci ratios: $1/1$ is 1. $2/1$ is 2. $3/2$ is 1.5. $5/3$ is 1.667. $8/5$ is 1.6. So these fractions are gradually settling down... but to what?

What happens to this ratio as we keep going to bigger Fibonacci numbers?

Coding challenge: Write a program to print out the first 50 Fibonacci ratios.

Advanced coding challenge: Do the same starting with numbers *other than* 0 and 1. For example start with -2 and 5. Compare the resulting ratio with the Fibonacci ratio result.



Graphing Grammar

This is a challenging coding project. It leads to many experiment ideas.

This Python code `s = "toothpaste"` places a string of letters into the variable `s`. This project begins with a very simple string. We modify it to make it longer, several times. Finally we read the result as *drawing instructions*. The drawing will be done using a Python Turtle. We will use two sets of rules. But before we get to that: Here is a warm-up: Write a program to input a string and then print every other letter in that string. Begin with this code:

```
s = input('Enter some text: ')
for c in s:
    print(c)
```

Next let us assign a string to a variable: `s = "x"`. To build a longer string from this we will use the first rule, called **Rule S**: Go through the string one character at a time and build a new string. When the character in `s` is "+" or "-": Copy it into the new string. If the character is "x" or "y": Replace it in the new sentence with these:

- "x" -> "y+x+y"
- "y" -> "x-y-x"

That is, every place you find 'x' replace it with 'y+x+y'. We did not need Python for that but we are now going to repeat the process, this time on our result string 'y+x+y' to make an even longer string: 'x-y-x+y+x+y+x-y-x'. What next? You guessed it: Repeat Rule S again and again and again, each time feeding in the result of the previous time. That is five times total. The end result will be about 500 characters.

Now to draw this using the second rule, Rule D. Go through the entire string one character at a time as before; but now ignore the letters x and y. Rule D says: For every "+" the Turtle turns left 60 degrees and goes forward 20. For every "-" the Turtle turns right 60 degrees and goes forward 20. Here is some code you may find useful.

```
t = Turtle()
for c in s:
    if c == '+': t.left(60)
    elif c == '-': t.right(60)
t.forward(20)
```

Once this is working it is time for you to experiment. You can change the forward amount, the angles, and you can change Rule S and Rule D.

Logic and time

Make your best guess: What does this code do? What will it print? Then run it to see.

```
from time import time, sleep

a = 1
b = 1

print("First logic check")
print(a == b)
if a == b: print(True)
else:      print(False)

a = 0
print("Second logic check")
print(a == b)
print(not (a == b))
print(not ((a == b) or not (a == b)))

# Now let's work with time

toc = time()
print(toc)
sleep(3)
tic = time()
print(tic)
print('elapsed time (seconds) =', tic - toc)
print(tic - toc > 0)

toc = time()
for i in range(1000000):
    j = i
tic = time()
million_time = tic - toc
print('It took me', str(million_time), 'seconds to count to one million')

seconds_per_minute = 60
minutes_per_hour = 60
hours_per_day = 24
days_per_month = 30
months_per_year = 12

# Fix this next line of code

seconds_per_year = 1

print("Number of seconds per year is", seconds_per_year)
```

Turtles in pursuit

This code features a Turtle chasing another Turtle. This is called *pursuit*. Below are two pursuit programs: Simple then fancier. After you understand how they work you can try to solve the Four Bugs problem on the next page.

```
# simple version of turtle pursuit
t = Turtle()
p = Turtle()
t.up()
t.goto(-300, -150)
t.down()
for c in range(400):
    t.forward(2)
    d = p.towards(t)
    p.setheading(d)
    p.forward(2)
```

```
from turtle import Turtle    # more complicated version
from math import cos

def r_speed(s):              # experiment: go faster when you get closer
    # if s < 35: return 4
    # if s < 78: return 3
    return 2

q = Turtle()                 # this turtle follows a winding path
r = Turtle()                 # this turtle chases after the first turtle
q.pencolor("blue")
r.pencolor("red")

q.up(); q.goto(-300, -150); q.down()
r.up(); r.goto(-100, 350); r.down()

for t in range(400):
    q.setheading(20 + 40*cos(.015*t))
    q.forward(2)
    r.setheading(r.towards(q))
    r.forward(r_speed(r.distance(q)))
```

Turtle Pursuit Challenge

Write a pursuit program to solve the Four Bugs Problem: Four bugs named A, B, C, and D begin at the 4 corners of a square table, side length 1 meter. Each bug faces the bug at the next corner going clockwise; for example D is facing towards A. At noon the bugs begin walking: Each bug walks towards the bug it is facing. They take identical tiny bug steps and they all move at the same speed. As they go they adjust the *direction* of their walking progress (Why?) Do the bugs eventually meet? If so: Where? And supposing they meet: How far has each of them walked?

Chaos Game

In the town of Monte Carlo there lives an artist named Egon. In the center of town is the town square, a large flat plaza paved entirely with bricks. Egon wished to paint something on the bricks of the town square but he could not think of what... until one day he found a single die (or "dice") that had escaped from a Yahtzee game. Egon found that when he rolled it the dice would come up 1, 2, 3, 4, 5 or 6 at random. And this is the idea he came up with for creating a painting.

- Egon set three empty paint cans A, B and C on the ground, far apart so as to mark the corners of a huge triangle. The cans are about 100 feet apart from each other.
- Egon walked to a random location on the town square where he could see his three paint cans, again marking the corners of a large triangle.
- Egon rolled the single dice (or "die") and followed these instructions:
 - On a 1 or a 2 he walks halfway to paint can A
 - On a 3 or a 4 he walks halfway to paint can B
 - On a 5 or a 6 he walks halfway to paint can C
 - He paints a dot on the ground at his feet
- From this new location Egon repeats the process:
 - He rolls the dice (or die) to select a paint can
 - He walks halfway to that paint can and paints a dot

Egon repeats this process many thousand times. What does the enormous painting look like?

Farmer Diophantus

There is a farmer named Diophantus: A very superstitious fellow who only likes integers. At the market he needs to buy animals for his farm. He has \$100. Cows cost \$10. Pigs cost \$3. Chickens cost \$0.50. He decides he must purchase exactly 100 animals with his 100 dollars... and must have at least one cow, one pig, and one chicken. What animals does he purchase?

Farmer Diophantus builds pens for his animal pens that are always right triangles with integer-length sides. He needs to build two pens. To save on fence material these pens will share one side. He finishes building the two triangular pens that share one side and finds he needed 37 meters of fence. What are the dimensions of the two pens?

Watering horses

Farmer Diophantus owns a number of horses; and he takes good care of them. They mostly live in a large corral, so large that he can't see it all from his house. On the far side he can see some trees and he thinks there is a stream running through it somewhere. Anyway he is not even certain how many horses there are living in this corral.

Here is a problem about getting water for some horses. In the corner of the horse corral near to his house there is a water trough. Each morning Farmer Diophantus finds some number of horses standing by this trough (which has a water pump) waiting for water because they are thirsty. Horses drink exactly one gallon each morning. If the number of horses is h then Farmer Diophantus must pour exactly h gallons of water into the trough, no more and no less. (Remember he is superstitious.)

Suppose there are four horses at the watering trough one morning ($h=4$). Farmer Diophantus has two water buckets on hand, and both are unmarked. The first contains three gallons when full, the second contains eight gallons when full. Because they are unmarked they can not be partially filled with any accuracy. He can't get four gallons, for example, by filling the eight gallon bucket half way. The buckets can only be completely filled, to three gallons or to eight gallons.

How does Farmer Diophantus pour exactly four gallons of water into the trough?

(To be sure: Farmer Diophantus is ok with pouring water out onto the ground as necessary.)

(There are not one but two solutions to this problem.)

Python challenge: Write a Python program that solves the horse watering problem: The program begins with the number of horses h and the size in gallons of the two containers a and b . The program gives a recipe for how to get to h gallons of water.

Number Cards

There are three cards on a table. Each has a unique positive integer written on it but the number side of the card is face down. Let us call the three numbers a , b and c . We are informed that that a is less than b and b is less than c and $a+b+c=13$. Now three girls (who know the conditions above) peek at the cards and make statements in the following order:

Anne says, after looking at a : I know a but I do not know b or c .

Caprice says, after looking at c : I know c but I do not know a or b .

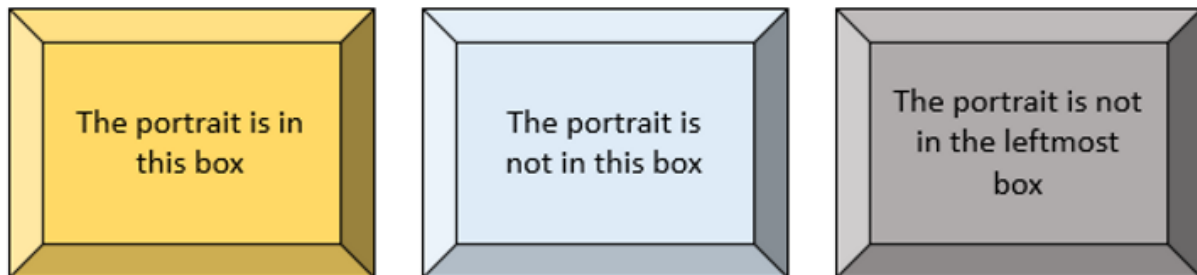
Betsy says, after looking at b : I know b but I do not know a or c .

What is b ?

Portia's Portrait

This is what we call an “unreliable narrator” problem. There are statements of fact that may be true or may be false. It is a good idea to develop your sense of when the narrator may be unreliable. In the challenge above about watering horses, for example: The narrator is perfectly reliable. In this problem matters are different.

Here are three boxes: One gold, one silver and one made of lead. A portrait of Portia (a famous historical figure) has been placed in one of the boxes; and statements are printed on the box lids as you can see.



Where is the portrait if at *most* one of these statements is true?

Dartboard pi

This is a Python challenge that I will describe very briefly. If you are puzzled by this and would like to know more please talk to the coaches at a Python Bytes meeting.

The area of a circle with radius = 1 is pi. The area of a square that just fits around this circle is 4. Suggestion: Draw a picture of this. Suppose in Python we make a square dartboard and throw darts at it. Each dart lands at a coordinate (x, y) where x is between -1 and 1 and so is y.

Suppose further that we throw 1000 darts and keep track of how many land inside the circle: Call this C. Some will land outside the circle; but probably most will land inside the circle. Now calculate the ratio C/1000. C is how many darts land inside the circle, 1000 is the number of darts total. What does this ratio mean? How could you use it to calculate the value of pi?

You can calculate whether a particular dart at (x, y) is inside the circle using the Pythagorean Theorem. A square-root function is available in the Python math library. Here is some test code:

```
from math import sqrt
from random import random
print(sqrt(25))
x = 2*random()-1
y = 2*random()-1
print(sqrt(x*x + y*y))
```

The Dragon! Another fractal challenge

Earlier in the packet was **Graphing Grammar**. This challenge is similar starting with a string I will call `s = '1'`. We use a new rule called Rule D many times to produce longer and longer strings from `s`. I will state Rule D and then work a few examples of Rule D.

Rule D: String `s` becomes the new string `new_s = s + '1' + s_prime...`

where `s_prime` is exactly the same as `s` except the middle character is flipped: If it is '1' in `s` it becomes '0' in `s_prime`; or if it is '0' in `s` it becomes '1' in `s_prime`.

Examples: We begin with `s = '1'`. Then Rule D tells us that `new_s` is '110'. Why? Because `s_prime` is the same as `s` with the center character flipped. There is only one character in `s` which is '1' so this is the center character and therefore `s_prime` is '0'. Rule D says that `new_s` is `s + '1' + s_prime` so this becomes '1' + '1' + '0' = '110'.

Now let's apply Rule D to this `new_s = '110'`. First let's rename it '`s`' :

```
s = '110'
new_s = '1101100'
```

Why? Because `s + '1' + s_prime` will be '110' + '1' + '100' (see the middle character of `s` is flipped in `s_prime`).

Last example: `s = '1101100'` so `new_s = '1101100' + '1' + '1100100' = '110110011100100'`. This then becomes '1101100111001001110110001100100'.

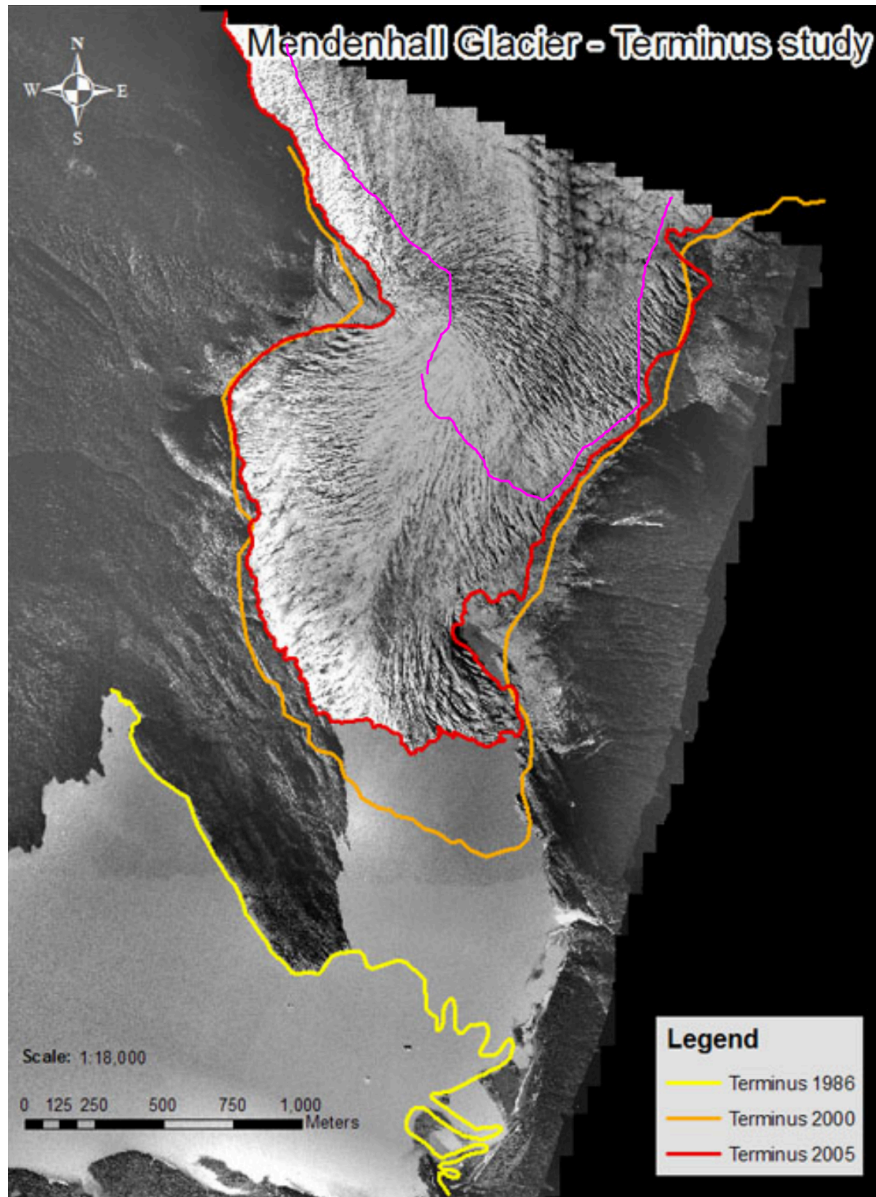
How to draw this with a Turtle? In **Graphing Grammar** we turned 60 degrees left or right. Here we turn 90 degrees left or right. Start with a loop to read each character in `s`: 'for `c` in `s`:' Each character `c` will be '1' or '0'. So first move forward 6 pixels: `t.forward(6)`. Then look at the next character `c` in the string `s`. If `c` is '0': turn left 90 degrees. If `c` is '1': turn right 90 degrees. Repeat for all the characters `c` in string `s`.

Spoiler alert: Here is some tricky Python code that makes `s_new` from `s`:

```
q          = len(s)
halfway    = q//2
middle     = s[halfway]
middle_flip = '0' if middle == '1' else '0'
s_prime    = s[:halfway] + middle_flip + s[halfway+1:]
s_new      = s + '1' + s_prime
```

Map Challenge

Here is a “real world” problem to think about. It is all about approximating so that our answer ends up *approximately* correct, we hope. The earth is warming and we can see the effects of this in many places including glaciers in Alaska.



The image above shows a glacier in 2005 outlined in red. It flows from the top of the image downward to end in a lake. The magenta line is where the glacier is today in 2025. The yellow line is where the glacier ended in 1986. Estimate what this melting from 1986 to today has contributed to sea level rise. Your answer should be in meters or if you like in centimeters. For reference the earth is 70% ocean and has a radius of 6378000 meters. Do the calculation using Python code (with variables). You will have to make some guesses!